

CHƯƠNG 2 - DOCKERFILE VÀ TỰ TẠO IMAGE

BÀI 1

TỪ SOURCE CODE ĐẾN DOCKER IMAGE

Mục tiêu của bài là hiểu quy trình từ source code và Dockerfile đến Image, trước khi bước sang phần thực hành viết Dockerfile.

1. Mục tiêu bài học

- Hiểu Dockerfile là gì và Dockerfile khác Image như thế nào.
- Hiểu docker build tạo Image, còn docker run tạo và chạy Container.
- Hiểu Base Image và vai trò của chỉ thị FROM.
- Hiểu Instruction trong Dockerfile.
- Hiểu Build Context và ý nghĩa dấu . trong docker build .
- Hiểu .dockerignore dùng để loại bỏ file không cần thiết khỏi Build Context.
- Biết sơ bộ về Image Layer và Build Cache.
- Hiểu tên Image và Tag khi tự build Image.

2. Từ sử dụng Image có sẵn đến tự tạo Image

Ở Chương 1, ta chủ yếu sử dụng Image do người khác tạo sẵn, ví dụ nginx. Docker tải Image từ Registry và tạo Container từ Image đó.

```
Docker Hub
↓
nginx Image
↓ docker run
nginx Container
```

Từ Chương 2, quy trình thay đổi. Ta sẽ tự chuẩn bị source code, viết Dockerfile và dùng Docker Build để tạo Image của chính mình.

```
Source Code
↓
Dockerfile
↓ docker build
Image của mình
↓ docker run
Container của mình
```

Ghi nhớ: Xương sống của Chương 2: Dockerfile -> docker build -> Image -> docker run -> Container.

3. Dockerfile là gì?

Dockerfile là file văn bản chứa các instruction mô tả các bước Docker phải thực hiện để xây dựng một Docker Image.

```
FROM nginx
COPY index.html /usr/share/nginx/html/index.html
```

Trong ví dụ trên, Docker được yêu cầu dùng nginx làm nền và đưa file index.html vào Image mới.

Lưu ý: Dockerfile không phải Image. Dockerfile chỉ mô tả cách tạo Image.

4. docker build là gì?

docker build là lệnh yêu cầu Docker xây dựng một Image dựa trên Dockerfile và Build Context.

```
docker build -t my-web:1.0 .
```

Kết quả của docker build là một Image, không phải Container.

Phân biệt build và run

```
BUILD
Dockerfile + Source Code
```

```
↓
docker build
↓
Image

RUN
Image
↓
docker run
↓
Container
```

Ghi nhớ: docker build tạo Image. docker run lấy Image làm đầu vào để tạo và chạy Container.

5. Base Image là gì?

Base Image là Image nền mà Image mới được xây dựng dựa trên đó.

```
FROM nginx
```

Dòng trên yêu cầu Docker dùng Image nginx làm nền. Sau đó ta có thể bổ sung file hoặc cấu hình của mình để tạo ra một Image mới.

```
nginx Image
+
index.html của mình
↓
my-web Image
```

Với ứng dụng .NET, ý tưởng cũng tương tự: dùng một Image đã có .NET Runtime hoặc .NET SDK làm nền rồi thêm ứng dụng của mình vào.

```
.NET Runtime Image
+
Product API
↓
Product API Image
```

Ghi nhớ: Base Image giúp ta không phải tự xây dựng toàn bộ môi trường chạy ứng dụng từ số 0.

6. Instruction trong Dockerfile

Mỗi dòng chỉ thị trong Dockerfile được gọi là một Instruction.

```
FROM nginx
COPY index.html /usr/share/nginx/html/
```

Trong ví dụ trên có hai instruction: FROM và COPY.

Các instruction phổ biến sẽ học dần trong Chương 2 gồm:

- FROM
- WORKDIR
- COPY
- RUN
- CMD
- ENTRYPOINT
- EXPOSE

Lưu ý: Không cần học thuộc toàn bộ instruction ngay ở Bài 1. Ta sẽ học đúng lúc khi thực hành.

7. Image Layer là gì?

Docker Image được cấu tạo từ nhiều layer. Nhiều instruction trong Dockerfile có thể tạo ra hoặc làm thay đổi các layer của Image.

```
Layer trên: file của ứng dụng
```

```
-----
```

```
Layer: cấu hình / dependency
```

```
-----
```

```
Base Image Layers
```

Các layer của Image về cơ bản là read-only. Khi Container được tạo, Docker thêm một writable layer riêng cho Container.

8. Build Cache là gì?

Build Cache là cơ chế giúp Docker tái sử dụng kết quả của những bước build trước đó nếu các bước đó không thay đổi.

```
Build lần 1  
Docker thực hiện các bước
```

```
Build lần 2  
Bước không đổi  
→ có thể dùng cache
```

Mục đích chính là giảm thời gian build và tránh lặp lại công việc không cần thiết.

Lưu ý: Ở bài này chỉ cần hiểu khái niệm. Cuối Chương 2 sẽ thực hành build lại để quan sát cache.

9. Build Context là gì?

Build Context là tập hợp file và thư mục mà Docker Build có thể truy cập trong quá trình xây dựng Image.

Ví dụ thư mục project:

```
E:\Docker\Projects\my-web\  
├─ Dockerfile  
├─ index.html  
└─ style.css
```

Nếu terminal đang đứng tại thư mục my-web và chạy:

```
docker build .
```

thì dấu . nghĩa là thư mục hiện tại, và Docker dùng thư mục hiện tại làm Build Context.

```
docker build .  
↑  
├─ thư mục hiện tại  
dùng làm Build Context
```

Khi Dockerfile chứa:

```
COPY index.html /usr/share/nginx/html/
```

Docker có thể lấy index.html vì file này nằm trong Build Context.

Ghi nhớ: Dấu . trong docker build . không phải viết cho có. Nó xác định Build Context.

10. Vì sao Build Context quan trọng?

Docker chỉ có thể sử dụng các file nằm trong Build Context khi thực hiện những thao tác như COPY. Nếu đúng sai thư mục hoặc chọn Build Context không phù hợp, Docker có thể không tìm thấy file cần thiết.

```
Project/
├── my-web/
│   ├── Dockerfile
│   └── index.html
└── other-files/
```

Do đó trước khi build cần biết terminal đang đứng ở đâu và Docker đang sử dụng thư mục nào làm Build Context.

11. .dockerignore là gì?

.dockerignore là file dùng để chỉ định các file hoặc thư mục Docker Build không cần đưa vào Build Context.

```
.git
bin
obj
*.log
```

Ví dụ với project có các thư mục .git, bin, obj hoặc file log, ta có thể loại chúng khỏi Build Context nếu chúng không cần cho quá trình build.

Lợi ích chính:

- Giảm kích thước Build Context.
- Giúp quá trình build gọn và nhanh hơn.
- Tránh đưa file không cần thiết vào quá trình build.
- Giảm nguy cơ vô tình đưa file nhạy cảm hoặc file tạm vào Image.

12. Image Name và Tag

Khi tự build Image, ta thường đặt tên và tag bằng tùy chọn -t.

```
docker build -t my-web:1.0 .
```

Trong đó:

- my-web: tên Image.
- 1.0: Tag.
- .: Build Context là thư mục hiện tại.

```
Source
↓
Dockerfile
↓
docker build -t my-web:1.0 .
↓
my-web:1.0 Image
↓
docker run
↓
Container
```

13. Project thực hành của Chương 2

Để tập trung vào Docker thay vì nghiệp vụ lập trình, project đầu tiên sẽ rất nhỏ:

```
my-web/  
├── Dockerfile  
└── index.html
```

Nội dung index.html có thể chỉ là:

```
Hello Docker
```

Dockerfile sau đó sẽ sử dụng Nginx làm Base Image và copy index.html vào Image.

```
FROM nginx  
COPY index.html /usr/share/nginx/html/index.html
```

Sau khi hiểu quy trình này, ta mới áp dụng cùng nguyên lý vào .NET API và cuối cùng là CRUD.

14. Các khái niệm cần nhớ

Dockerfile

File mô tả cách Docker xây dựng Image.

Base Image

Image nền dùng làm điểm bắt đầu để tạo Image mới.

Instruction

Chỉ thị trong Dockerfile, ví dụ FROM hoặc COPY.

docker build

Lệnh xây dựng Image.

Build Context

Các file và thư mục Docker Build có thể sử dụng.

Image Layer

Các lớp tạo thành Docker Image.

Build Cache

Cơ chế tái sử dụng kết quả các bước build không thay đổi.

.dockerignore

File loại các file/thư mục không cần thiết khỏi Build Context.

Tag

Nhãn giúp xác định Image hoặc phiên bản Image.

15. Câu hỏi kiểm tra và đáp án

1. Dockerfile có phải Image không?

Không. Dockerfile là file mô tả cách build Image.

2. docker build tạo ra Image hay Container?

docker build tạo ra Image.

3. docker run nhận đầu vào chính là Dockerfile hay Image?

docker run sử dụng Image để tạo và chạy Container.

4. Base Image dùng để làm gì?

Base Image là Image nền để xây dựng Image mới. Nó cung cấp môi trường ban đầu như hệ thống file, runtime hoặc phần mềm nền cần thiết.

5. Trong docker build ., dấu . có ý nghĩa gì?

Dấu . nghĩa là thư mục hiện tại và Docker dùng thư mục đó làm Build Context.

6. Vì sao Docker cần .dockerignore?

.dockerignore dùng để loại các file/thư mục không cần thiết khỏi Build Context, giúp build gọn hơn, nhanh hơn và hạn chế đưa file không mong muốn vào quá trình build.

16. Tóm tắt cuối bài

Source Code

↓

Dockerfile

↓

docker build

↓

Image

↓

docker run

↓

Container

Base Image là nền của Image mới.

Build Context là vùng file Docker Build được phép sử dụng.

.dockerignore loại bớt file không cần thiết khỏi Build Context.

Build Cache giúp tái sử dụng các bước build không thay đổi.

-t dùng để đặt tên và tag cho Image.

Bài tiếp theo: Chương 2 - Bài 2: Tạo project my-web đầu tiên